Part 1 - Pygame Car Crash Tutorial

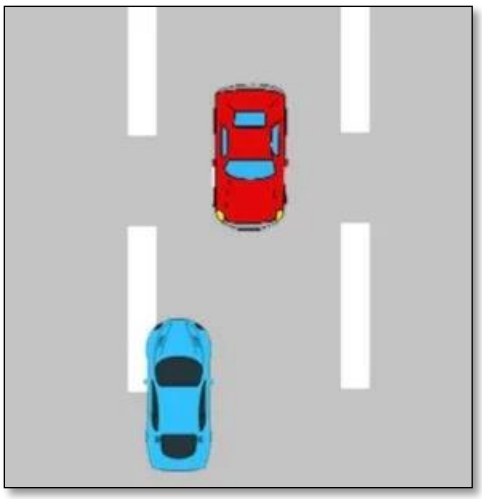
Contents

Part 1 - Pygame Car Crash Tutorial....1
Preview of the Game....1
Content....2
CarCrash Class....2
Classes and Objects....2
Game Loop....3
CarCrash Game Loop....5
blit....6
Assignment Submission....6

Time required: 30 minutes

Preview of the Game

Here's a sneak peak of the game that we are going to work on.



Car Crash is simple arcade type game. The object is to move your blue car back and forth to avoid the oncoming red cars.

Content

There is a **Content.zip** file attached to the assignment. This has all the images and sounds we will use in the game.

1. Create a folder named **content** in your game directory.
2. Extract the files and copy them to the **content** folder.

CarCrash Class

The first step is to draw the game without any motion. We will start with displaying the background image.

1. At a command prompt install the pygame library: **pip install pygame**
2. Create a new Python file. Save it as **car_crash_1.py**
3. Add the following code.

```
"""
    Name: car_crash_1.py
    Author:
    Date:
    Purpose: Draw the background
"""
# pip install pygame
# Import pygame and sys modules
import pygame
from sys import exit
```

At the top of the code we import the **pygame** module and **exit** from the **sys** module.

Classes and Objects

We'll be using Classes and Objects for our program. Whether it's GUI, Pygame or any other large application the Object-Oriented Programming approach is almost always the best idea. Using Classes, we'll be using methods to store blocks of code that are to be repeated several times throughout the game.

```python
class CarCrash:
    # Setup class constants
    SURFACE_WIDTH = 400
    SURFACE_HEIGHT = 600

    def __init__(self):
        # Initialize pygame for action
        pygame.init()

        # Setup computer clock object to control the speed of the game
        self.CLOCK = pygame.time.Clock()

        # Create the game surface (window)
        self.surface = pygame.display.set_mode(
            (self.SURFACE_WIDTH, self.SURFACE_HEIGHT)
        )

        # Load background image from file
        self.background = pygame.image.load("content/street.png")

        # Set window caption
        pygame.display.set_caption("Car Crash")
```

1. Create the CarCrash class. The class constants define the width and height of the game.
2. The **__init__** method creates the CarCrash class.
3. **pygame.init()** initializes the pygame library.
4. **self.CLOCK** is used to control the speed of the game.
5. Set the display mode using **pygame.display.set_mode** to create a window (drawing surface). The window dimensions are defined by **SURFACE_WIDTH** and **SURFACE_HEIGHT.** This is stored in the **self.surface** variable.
6. Load the street.png from file into an image variable.
7. Set the window caption.

Game Loop

A game loop is a fundamental structure in game development that continuously repeats, or loops, to handle various tasks and update the game state. It's the heart of a video game,

responsible for coordinating and executing the sequence of actions that make up the gameplay. The main components of a typical game loop include:

1. **Input**
    a. Gather input from the player, such as keyboard, mouse, or controller inputs.
2. **Update**
    a. Update the game state and entities based on the input and the passage of time.
    b. This includes handling player movement, updating AI behavior, checking for collisions, and managing other aspects of the game logic.
3. **Render**
    a. Draw or render the current state of the game to the screen.
    b. This involves updating the graphics, animations, and any visual elements that the player interacts with.
4. **Timing**
    a. Manage the timing of the loop to ensure a consistent frame rate.
    b. This helps in creating a smooth and responsive gaming experience.

The loop continues to iterate, repeating these steps, creating the illusion of continuous motion and interaction. The game loop runs as long as the game is active, providing real-time updates and responses to player input.

Here's a simplified example in pseudo-code:

```
while game_is_active:
    # Process player input
    handle_input()

    # Update game state
    update_game_state()

    # Render the current state
    render()

    # Control the frame rate and wait if needed
    control_frame_rate()
```

The exact implementation of a game loop may vary based on the game engine or framework being used, but the fundamental structure remains similar across most games.

CarCrash Game Loop

Add the following code to your **car_crash_1.py** file.

```python
# ---------------------- RUN GAME ---------------------------------------#
    def run_game(self):
        """Start the infinite Game Loop"""
        while True:
            # Change in delta time for each frame
            # Multiplying movement by dt allows all computers
            # to run the game at the same speed
            # 100 frames per second
            self.dt = self.CLOCK.tick(100)

            # Listen for all window events
            for event in pygame.event.get():

                # Closing the program by clicking the X
                # causes the QUIT event to be fired
                # Exit game if window is closed
                if event.type == pygame.QUIT:
                    # Quit Pygame
                    pygame.quit()
                    # Exit Python
                    exit()

        # ----------------- DRAW ON BACKBUFFER -------------------------#
            # Draw everything on the backbuffer first
            # Fill the surface with the background image loaded earlier
            self.surface.blit(self.background, (0, 0))

        # ----------------- UPDATE SURFACE -----------------------------#
            # From backbuffer, update Pygame display to reflect any changes
            pygame.display.update()
```

The commands shown above are all in the game loop, they repeat continuously.

1. Track the change in delta time for each frame.
2. Get any Pygame events. If we close the program window, the program exits.
3. Refresh and clear the surface using the blit function.

4. The **pygame.display.update()** command updates the screen with any changes to the backbuffer that have occurred up until this point.

blit

In Pygame, the term "blit" stands for Block Transfer. It refers to the process of copying the pixel values from one surface onto another. In the context of game development or graphical applications using Pygame, the **blit** method is commonly used to draw images onto a screen or another surface.

```
# ------------------ DRAW ON BACKBUFFER ---------------------------#
    # Draw everything on the backbuffer first
    # Fill the surface with the background image loaded earlier
    self.surface.blit(self.background, (0, 0))
```

- **self.surface** - The destination surface onto which you want to draw the image.
- **self.background** - The surface containing the image you want to draw.
- **(x, y)** – The coordinates specifying where on the destination surface the top-left corner of the source image should be placed. (0, 0) is in the upper left corner of the screen.

Assignment Submission

Zip up the program files folder and submit in Blackboard.